

intro to python

Credit Hours: 3-0

Course Outline

Introduction to python language

16-Week Course Plan

Week 1

****Week 1: Introduction to Python (3-0)****

Lecture 1.1: Introduction to Python

Learning Objectives

- * Understand the basics of Python programming language
- * Identify the features and benefits of Python

Concepts

Python is a high-level, interpreted programming language. It was created in the late 1980s by Guido van Rossum. Python is known for its simplicity, readability, and ease of use. It is a general-purpose language, meaning it can be used for a wide range of applications, including web development, scientific computing, and data analysis.

Lecture 1.2: Basic Syntax and Data Types

Learning Objectives

- * Understand the basic syntax of Python
- * Identify the basic data types in Python

Concepts

Python uses indentation to define code blocks. The basic syntax includes variables, data types, operators, and control structures. Python has several basic data types, including integers, floats, strings, and booleans.

Lecture 1.3: Installing Python and IDEs

Learning Objectives

- * Understand how to install Python on a computer
- * Identify popular Integrated Development Environments (IDEs) for Python

Concepts

Python can be installed on Windows, macOS, or Linux. Popular IDEs for Python include PyCharm, Visual Studio Code, and Spyder. These tools provide features such as code completion, debugging, and project management.

Week 2

****Week 2: Control Structures****

****3-0****

Lecture 1: Conditional Statements

Conditional statements are used to execute different blocks of code based on a condition. They are crucial in decision-making processes in programming.

Learning Objectives:

- Understand the syntax and usage of if-else statements
- Learn about the different types of comparison operators in Python
- Apply conditional statements in simple programs

Lecture 2: Conditional Statements (continued)

- ****if-elif-else statements****: Used for multiple conditions
- ****else statement****: Used when no condition is met
- ****Comparison operators****: ==, !=, >, <, >=, <=, etc.

Lecture 3: Loops

Loops allow us to execute a block of code repeatedly for a specified number of times. They are essential in repetitive tasks in programming.

Learning Objectives:

- Understand the syntax and usage of for loops
- Learn about the different types of loops in Python
- Apply for loops in simple programs

Week 3

****Week 3: Conditional Statements (3-0)****

Lecture Title: Conditional Statements

Learning Objectives:

- Understand the concept of conditional statements
- Learn how to use if-else statements in Python
- Understand the syntax and structure of if-else statements

Concepts:

Conditional statements are used to execute different blocks of code based on a condition. In Python, the if-else statement is used for this purpose. The syntax of the if-else statement is as follows:

```
'''
```

```
if condition:
```

```
# code to be executed if condition is true
```

else:

code to be executed if condition is false

...

The condition can be a boolean expression, a comparison, or a logical operation. The code inside the if block is executed only when the condition is true. If the condition is false, the code inside the else block is executed.

Example:

...

x = 5

if x > 10:

print("x is greater than 10")

else:

print("x is less than or equal to 10")

...

This code will output: "x is less than or equal to 10" because the condition x > 10 is false.

Additional Concepts:

- Nested if-else statements
- elif statement (else if)

Key Terms:

- Conditional statements
- If-else statement
- Boolean expression
- Logical operation

Week 4

****Week 4: Functions and Modules (3-0)****

Lecture 1: Introduction to Functions

Learning Objectives

- Define a function in Python
- Understand the purpose of functions
- Identify the benefits of using functions

Concepts

A function is a block of code that can be executed multiple times from different parts of a program. It takes arguments, performs a specific task, and returns a value. Functions help organize code, reduce repetition, and improve code readability.

Lecture 2: Function Syntax and Arguments

Learning Objectives

- Understand the syntax of a function definition
- Learn how to pass arguments to a function
- Identify the difference between positional and keyword arguments

Concepts

A function definition starts with the `def` keyword followed by the function name and a list of arguments in parentheses. Arguments can be passed in a specific order (positional) or using a keyword (keyword).

Lecture 3: Function Return Values and Modules

Learning Objectives

- Understand how functions return values
- Learn how to import modules in Python
- Identify the difference between built-in and user-defined modules

Concepts

Functions can return values using the `return` statement. Modules are files that contain a collection of related functions, classes, or variables. Modules can be imported using the `import` statement and used in a program.

Week 5

****Week 5: Functions and Modules (3-0)****

Lecture Title: Functions in Python

Learning Objectives

- Define a function in Python
- Explain the concept of arguments and return values in functions
- Use functions to organize code and improve reusability

Concepts

In Python, a function is a block of code that can be executed multiple times from different parts of a program. Functions help to organize code, make it reusable, and improve readability.

A function in Python is defined using the `def` keyword followed by the function name and a list of parameters in parentheses. The function body is indented under the function definition.

```

python

def greet(name):

print("Hello, " + name + "!")

greet("John")


```

Functions can take arguments, which are values passed to the function when it is called. Functions can also return values, which are sent back to the caller after the function completes execution.

Functions can be used to perform complex tasks, such as data processing, file operations, and web requests.

Lecture Content

- Functions as reusable code blocks
- Function arguments and return values
- Using functions to improve code organization and reusability

Lecture Notes

- * Define a function in Python using `def` keyword
- * Use arguments and return values in functions
- * Explain the benefits of using functions in Python programming

Reading Assignment

- * Chapter 5: Functions in Python (pages 123-145)
- * Section 5.1: Defining Functions (pages 123-125)
- * Section 5.2: Function Arguments and Return Values (pages 126-130)

Quiz Questions

1. What is the purpose of defining a function in Python?
2. How do you pass arguments to a function in Python?
3. What is the benefit of using functions in Python programming?

Homework

- * Write a Python function to calculate the area of a rectangle
- * Use a function to solve a real-world problem (e.g., calculating the tip in a restaurant)

Week 6

Week 6: Functions in Python

=====

Lecture Title

Functions in Python

Learning Objectives

- * Understand the concept of functions in Python
- * Learn how to define and use functions in Python
- * Understand function arguments, return types, and scope

Concepts

Defining Functions

In Python, a function is defined using the `def` keyword followed by the function name and a list of parameters in parentheses.

```
```python
def greet(name):
 print("Hello, " + name + "!")
...

```

### #### Function Arguments

Functions can take in arguments, which are values passed to the function when it's called.

```
```python
def greet(name):
    print("Hello, " + name + "!")
greet("John")
...

```

Function Return Types

Functions can return values using the `return` statement.

```
```python
def add(a, b):
 return a + b
result = add(3, 5)
print(result) # Outputs: 8
...

```

### #### Function Scope

Functions have their own scope, which means variables defined inside a function are not accessible outside the function.

```

```python
def outer():
    x = 10
    def inner():
        print(x)
    inner()
    # print(x) # This would raise a NameError
outer()
```

```

### **Lecture Notes**

- \* Functions help organize code and make it reusable.
- \* Functions can take in arguments and return values.
- \* Functions have their own scope, which helps prevent variable conflicts.

## **Week 7**

**\*\*Week 7: Functions in Python\*\***

**\*\*3-0\*\***

## **Lecture 1: Introduction to Functions**

### **Learning Objectives**

- \* Define functions in Python
- \* Understand the purpose of functions
- \* Identify the benefits of using functions

### **Concepts**

Functions in Python are blocks of code that can be executed multiple times from different parts of the program. They are defined using the `def` keyword followed by the function name and parameters in parentheses. Functions can take arguments, perform operations, and return values. They are useful for organizing code, reducing repetition, and improving modularity.

## **Lecture 2: Function Arguments and Return Values**

### **Learning Objectives**

- \* Understand function arguments
- \* Understand function return values
- \* Identify the difference between positional and keyword arguments



## Concepts

Functions can take arguments, which are values passed to the function when it is called. These arguments can be positional or keyword arguments. Positional arguments are passed in the order they are defined in the function, while keyword arguments are passed using the argument name. Functions can also return values, which can be used in the calling code. The `return` statement is used to specify the value returned by a function.

## Lecture 3: Function Examples and Best Practices

### Learning Objectives

- \* Understand how to write a simple function
- \* Identify best practices for function design
- \* Learn how to use the `print` function to debug code

## Concepts

A simple function can be defined to perform a specific task, such as calculating the area of a rectangle. Functions should be designed to be reusable and easy to understand. The `print` function can be used to debug code by printing the values of variables and function outputs. Functions can also be used to encapsulate complex logic and make code more modular.

## Week 8

**Week 8:**

### Lecture Title: **Exception Handling in Python**

#### Learning Objectives:

- \* Understand the concept of exceptions in Python
- \* Learn how to use try-except blocks to handle exceptions
- \* Understand the difference between built-in and custom exceptions

#### Concepts:

Python provides a built-in exception handling mechanism to handle runtime errors. An exception is an event that occurs during the execution of a program and disrupts the normal flow of instructions. Python uses try-except blocks to handle exceptions.

#### #### Try Block

The try block contains the code that may potentially raise an exception. The code within the try block is executed first.

```
```python
```

```
try:
```

```
# code that may raise an exception
```

```
x = 5 / 0
```

```
except:
```

```
# exception handling code
```

```
print("An error occurred")
```

```
...
```

```
##### Except Block
```

The except block contains the code that is executed when an exception occurs. The except block can specify the type of exception it can handle.

```
```python
```

```
try:
```

```
code that may raise an exception
```

```
x = 5 / 0
```

```
except ZeroDivisionError:
```

```
exception handling code
```

```
print("Cannot divide by zero")
```

```
...
```

```
Raising Custom Exceptions
```

Python allows us to raise custom exceptions using the raise keyword.

```
```python
```

```
raise ValueError("Invalid input")
```

```
...
```

Practice:

- * Try-except blocks
- * Built-in exceptions (e.g., TypeError, ValueError)
- * Custom exceptions
- * Raising exceptions using the raise keyword

Additional Resources:

- * Python documentation on exceptions:

****Total Lecture Hours: 3, Lab Hours: 0****

This week's lecture focuses on exception handling in Python, including try-except blocks, built-in exceptions, custom exceptions, and raising exceptions using the raise keyword. Students will learn how to write try-except blocks to handle exceptions and understand the difference between built-in and custom exceptions.

Week 9

****Week 9: Exception Handling and Debugging****

Lecture Title

****Error Handling and Debugging in Python****

Learning Objectives

- Understand the concept of exceptions in Python
- Learn how to handle exceptions using try-except blocks
- Understand how to use the debugger in Python

Concepts

In Python, exceptions are events that occur during the execution of a program that disrupt the normal flow of instructions. Exception handling is a mechanism to handle these exceptions and prevent the program from crashing. We use try-except blocks to handle exceptions. The try block contains the code that may raise an exception, and the except block contains the code that will be executed if an exception occurs. We can also use the finally block to execute code regardless of whether an exception occurred or not. The debugger in Python is a powerful tool that allows us to step through our code line by line and examine variables and their values.

****Example Code****

```
```python
try:
 x = 5 / 0
except ZeroDivisionError:
 print("Cannot divide by zero!")
```
```

****Key Points****

- * Use try-except blocks to handle exceptions
- * Use the finally block to execute code regardless of whether an exception occurred or not
- * Use the debugger to step through your code and examine variables and their values

****Lab (1 hour)****

- * Implement exception handling in a Python program
- * Use the debugger to step through your code and examine variables and their values
- * Write a Python program that handles multiple exceptions using a single try-except block

****Assessment****

- * Complete the lab exercise

* Submit your code for review

Week 10

****Week 10: Exception Handling and Error Management (3-0)****

Lecture Title

Exception Handling and Error Management in Python

Learning Objectives

- Understand the purpose of exception handling in Python.
- Learn how to use try-except blocks to handle exceptions.
- Develop skills to create custom exceptions.

Lecture Content

Python provides a robust system for handling exceptions and errors. Exception handling is a mechanism to handle runtime errors that occur during the execution of a program. The try-except block is a fundamental construct used for exception handling. The `try` block contains the code that may potentially raise an exception, while the `except` block contains the code that will be executed when an exception occurs.

```
```python
try:
 # Code that may raise an exception
 x = 5 / 0
except ZeroDivisionError:
 # Code to handle the exception
 print("Error: Division by zero is not allowed.")
```
```

In addition to built-in exceptions, Python allows developers to create custom exceptions by creating a subclass of the `Exception` class.

```
```python
class InsufficientBalanceError(Exception):
 pass

try:
 # Code that may raise an exception
 balance = 100
 if balance < 1000:
 raise InsufficientBalanceError("Insufficient balance.")
```
```

```
except InsufficientBalanceError as e:
```

```
# Code to handle the exception
```

```
print(e)
```

```
```
```

### Concepts

- Try-except block
- Custom exceptions
- Exception hierarchy in Python

### Lecture Summary

This lecture covered the basics of exception handling in Python, including try-except blocks, custom exceptions, and the exception hierarchy. Students should be able to understand the importance of exception handling and know how to implement it in their code.

## Week 11

**Week 11: Exception Handling**

### Lecture Title

Exception Handling in Python

### Learning Objectives

- \* Understand the concept of exceptions in Python
- \* Learn how to handle exceptions using try-except blocks
- \* Understand the different types of exceptions in Python

### Concepts

Python provides a mechanism to handle runtime errors using exceptions. An exception is an event that occurs during the execution of a program that disrupts the normal flow of instructions.

There are two types of exceptions:

\* **Syntax Errors**: errors that occur during the compilation phase, such as missing or mismatched parentheses.

\* **Runtime Errors**: errors that occur during the execution phase, such as division by zero or out-of-range values.

To handle exceptions, we use try-except blocks. The try block contains the code that might raise an exception, and the except block contains the code that will be executed if an exception is raised.

**Example**

```
```python
```

```
try:
x = 5 / 0
except ZeroDivisionError:
print("Cannot divide by zero!")
...
```

In this example, the try block attempts to divide 5 by 0, which raises a ZeroDivisionError. The except block catches this exception and prints an error message.

Week 12

****Week 12: Exceptions and Error Handling****

Lecture Title: Handling Exceptions in Python

Learning Objectives

- * Understand the concept of exceptions in Python
- * Learn how to raise and handle exceptions using try-except blocks
- * Use the `raise` statement to create custom exceptions

Concepts

In Python, an exception is an event that occurs during the execution of a program that disrupts the normal flow of instructions. Exceptions can be triggered by various factors, such as division by zero, type errors, or file not found errors.

To handle exceptions, Python provides the `try`-`except` block structure. The `try` block contains the code that might raise an exception, while the `except` block contains the code that handles the exception.

```
```python
try:
Code that might raise an exception
x = 5 / 0
except ZeroDivisionError:
Handle the exception
print("Cannot divide by zero!")
...
```
```

You can also use the `raise` statement to create custom exceptions.

```
```python
class InsufficientFundsError(Exception):
pass
```
```

```

try:
# Code that might raise an exception
raise InsufficientFundsError("Insufficient funds in account.")
except InsufficientFundsError as e:
# Handle the exception
print(e)
...

```

Key Takeaways

- * Use `try`-`except` blocks to handle exceptions in Python
- * Raise custom exceptions using the `raise` statement
- * Handle specific exceptions using the `except` block

This week's lecture focuses on understanding and handling exceptions in Python, which is a crucial aspect of writing robust and error-free code. By mastering exception handling, you'll be able to write more reliable and efficient Python programs.

Week 13

****Week 13: Exception Handling and Error Handling****

Lecture Title

Exception Handling and Error Handling

Learning Objectives

- * Understand the concept of exceptions and error handling in Python
- * Learn how to handle exceptions using try-except blocks
- * Know how to use built-in exception classes and custom exception classes

Concepts

In Python, exceptions are events that occur during the execution of a program that disrupt the normal flow of instructions. Error handling is the process of handling exceptions to prevent the program from crashing.

Try-Except Blocks

A try-except block is a block of code where you try to execute a statement and catch any exceptions that may occur. The general syntax is:

```

```python
try:
code to be executed

```

```
except ExceptionType:
```

```
code to handle the exception
```

```
...
```

#### #### Built-in Exception Classes

Python has several built-in exception classes that can be raised, such as:

- \* ``ValueError``: raised when a function receives an argument with an incorrect value

- \* ``TypeError``: raised when a function receives an argument of the wrong type

- \* ``ZeroDivisionError``: raised when a division by zero occurs

#### #### Custom Exception Classes

You can also create your own custom exception classes by inheriting from the ``Exception`` class.

### Example

Here's an example of using a try-except block to handle a ``ValueError``:

```
```python
```

```
def divide(a, b):
```

```
    try:
```

```
        result = a / b
```

```
    return result
```

```
    except ValueError:
```

```
        print("Cannot divide by zero!")
```

```
    except TypeError:
```

```
        print("Invalid input type!")
```

```
```
```

### Lab

Create a program that demonstrates exception handling using try-except blocks and built-in exception classes. (0 lab hours)

## Week 14

**\*\*Week 14: Exception Handling in Python\*\***

### Lecture Title

Exception Handling in Python

### Learning Objectives

- \* Understand the concept of exceptions in Python



- \* Learn how to raise and handle exceptions
- \* Use try-except blocks to handle errors

## Concepts

Python provides a way to handle runtime errors using exceptions. An exception is an event that occurs during the execution of a program that disrupts the normal flow of instructions.

### **\*\*Raising Exceptions\*\***

```
```python
raise ValueError("Invalid input")
```
```

We can raise a custom exception using the `raise` keyword.

### **\*\*Handling Exceptions\*\***

```
```python
try:
    # code that might raise an exception

except ValueError as e:
    # handle the exception
    print(f"Error: {e}")
```
```

The `try` block contains the code that might raise an exception. The `except` block contains the code that handles the exception.

### **\*\*Multiple Except Blocks\*\***

```
```python
try:
    # code that might raise an exception

except ValueError as e:
    # handle ValueError exception
    print(f"Error: {e}")

except TypeError as e:
    # handle TypeError exception
    print(f"Error: {e}")
```
```

We can have multiple `except` blocks to handle different types of exceptions.

## **\*\*Best Practices\*\***

- \* Always handle exceptions to prevent the program from crashing
- \* Provide meaningful error messages to help with debugging
- \* Use specific exception types to handle specific errors

This concludes the lecture on exception handling in Python.

## **Week 15**

### **\*\*Week 15: Exception Handling and Error Management in Python\*\***

#### **Lecture Title: Exception Handling in Python**

#### **Learning Objectives**

- Understand the concept of exceptions in Python
- Learn to use try-except blocks to handle exceptions
- Understand the different types of exceptions in Python

#### **Concepts**

In Python, exceptions occur when an error happens during the execution of a program. These exceptions are handled using try-except blocks. A try block contains code that may potentially raise an exception, and an except block contains code that will be executed if an exception occurs.

#### **\*\*Example:\*\***

```
```python
try:
    x = 5 / 0
except ZeroDivisionError:
    print("Error: Division by zero is not allowed")
```
```

In this example, the try block attempts to divide 5 by 0, which raises a ZeroDivisionError. The except block catches this exception and prints an error message.

#### **\*\*Types of Exceptions:\*\***

- Built-in exceptions: These are exceptions that are built into the Python interpreter, such as ZeroDivisionError, TypeError, and ValueError.
- User-defined exceptions: These are exceptions that are defined by the user using the raise statement.

#### **Lecture Title: Error Management in Python**

#### **Learning Objectives**

- Understand the concept of error management in Python
- Learn to use error messages to diagnose and fix errors
- Understand the importance of error handling in large programs

### Concepts

Error management is an essential aspect of programming in Python. It involves handling errors and exceptions that occur during the execution of a program. Error messages provide valuable information that can be used to diagnose and fix errors.

**\*\*Best Practices:\*\***

- Always include error messages when raising exceptions
- Use try-except blocks to handle exceptions
- Debug your code to identify and fix errors

**\*\*Assignment:\*\***

Implement exception handling in a Python program to handle potential errors.

## Week 16

**\*\*Week 16: Exception Handling and Debugging\*\***

### Lecture Title: Exception Handling and Debugging in Python

#### Learning Objectives

- \* Understand the concept of exceptions in Python
- \* Learn how to handle exceptions using try-except blocks
- \* Understand how to debug Python code

### Concepts

In Python, exceptions are events that occur during the execution of a program, such as division by zero or attempting to access an index out of range. Exception handling allows you to write code that can handle these exceptions and prevent the program from crashing.

A try-except block is used to handle exceptions in Python. The try block contains the code that might raise an exception, and the except block contains the code that will be executed if an exception is raised.

```
```python
try:
    # Code that might raise an exception
    x = 5 / 0
except ZeroDivisionError:
    # Code to handle the exception
```

```
print("Cannot divide by zero!")
```

```
...
```

Python also provides a built-in debugger that can be used to step through code line by line and examine variables. The ``pdb`` module can be used to debug code.

```
```python
```

```
import pdb
```

```
pdb.set_trace()
```

```
...
```

This will stop the execution of the code at the current line and allow you to step through the code using commands such as ``n`` (next line), ``s`` (step into a function), and ``c`` (continue execution).

**\*\*No lab required for this week.\*\***